

AI OPERATIONS PORTFOLIO · USER MANUAL

CS2 Analytics Engine

An end-to-end data pipeline: raw game-replay files in, a coached performance report out.

John White

AI Operations & Optimization

[linkedin.com/in/johnwhiteimse](https://www.linkedin.com/in/johnwhiteimse) · github.com/johnwhite-ai/ai-operations-portfolio

Contents

- 01** Executive Summary
- 02** The Problem
- 03** How It Works
- 04** The Analytical Depth
- 05** Design Rationale & Tradeoffs
- 06** What This Project Demonstrates

01 Executive Summary

The CS2 Analytics Engine is an end-to-end analytics pipeline that ingests raw game-replay files and automatically produces a sourced performance report with specific, prioritized recommendations. It parses thousands of in-game events and positional snapshots, computes metrics across five analytical dimensions, and translates the findings into plain-language coaching.

It is a personal project built end to end — and it demonstrates the exact pattern that underpins professional operations work: raw, messy data in, an automated and sourced decision out.

THE ONE-LINE VERSION

Every match produces the data needed to improve, locked in a dense binary format. This engine turns it into "here is your specific weakness and here is what to practice" — automatically, every run.

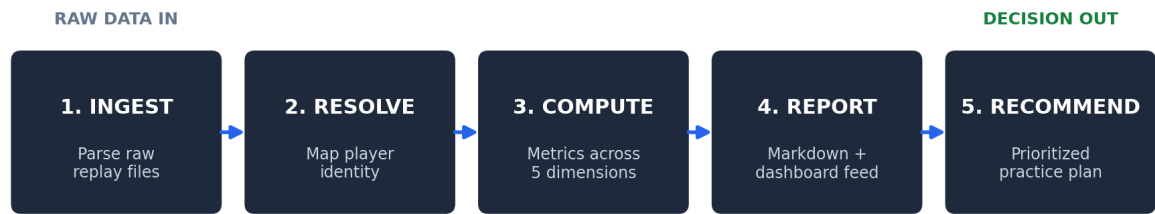
02 The Problem

Improving at a complex competitive game means knowing specifically what you did wrong, not getting generic advice. The raw data exists inside every match replay file, but it is locked in a dense binary format. Turning it into actionable feedback requires parsing thousands of events and positional ticks, computing the right metrics, and translating them into recommendations.

This is the universal operations problem in miniature: raw, messy data that contains the answer, with no automated path from data to decision.

03 How It Works

A multi-stage pipeline that runs with a single command.



CS2 Analytics Engine · fully automated, single command

The five-stage pipeline: ingest, resolve, compute, report, recommend — raw data in, a decision out.

Stage	What it does
Ingest	Parse raw replay files; extract kill/death events, weapon data, positional ticks for every player and round
Resolve	Map a display name to a canonical player ID across multiple matches so stats aggregate correctly
Compute	Metrics in five dimensions: mechanical, positional, behavioral, situational, map-specific
Report	Produce a clean markdown report plus a JSON sidecar that feeds a web dashboard
Recommend	Translate the computed weaknesses into specific, prioritized practice recommendations

04 The Analytical Depth

The engine does not just compute surface stats. It builds five tiers of analysis from the raw data:

- **Mechanical** — K/D, headshot %, per-weapon and per-side accuracy.
- **Positional** — nearest-teammate distance, close-support vs. lone-wolf rate, computed from raw tick data.
- **Behavioral** — survival rate, first-death rate, trade participation, pre-aimed-death rate.
- **Situational** — clutch performance by scenario, enemy-context heuristics.
- **Map-specific** — per-map performance to isolate where mechanics break down.

FROM NUMBERS TO A DECISION

The output is not a dashboard of statistics. It is a prioritized list of specific practice recommendations, each tied to a computed weakness — the analytical equivalent of an operations report that ends with "here is what to do about it."

05 Design Rationale & Tradeoffs

Why build a thin slice instead of a full platform?

Existing tools already cover the basic dashboard layer. The high-leverage gap was the feedback loop between "what I actually did wrong" and "what I should practice next" — so the engine was scoped to that slice and deliberately did not rebuild the commodity dashboard. That scope discipline is itself a demonstrated skill.

Why positional tick analysis?

Event data (kills, deaths) is easy but shallow. The genuinely useful signals — am I over-isolated, am I peeking into pre-aimed angles — only emerge from positional tick data, which is far harder to work with. Building that tier is what separates real analysis from a stats readout.

06

What This Project Demonstrates

Capability	How this project proves it
Data pipeline engineering	Raw binary files to parsed events to computed metrics to report, end to end
Turning data into decisions	Output is prioritized recommendations, not just numbers
Sourced, trustworthy output	Every metric traces back to the raw data with no manual entry
Scope discipline	Built the high-leverage analytics slice; deferred lower-value work deliberately
Full-stack delivery	Python analysis, a FastAPI service, and a web dashboard
Initiative	Built independently, end to end, for personal use

THE TAKEAWAY

This is the same pattern applied to operations work: take raw, messy data and build the system that turns it into a decision someone can act on.